

MATLAB TUTORIAL

This tutorial gives a set of commands that illustrate how to create a wheel movement signal $\{t, \alpha\}$ and how to use that to generate a set of quadrature output signals **A** and **B**.

Type in the following commands in the matlab command window to see how to use the instructor-provided matlab functions for use with this project.

A. Instructor-provided matlab functions

Work through the following code to develop some familiarity with the instructor-provided matlab functions.

A.1 Define the wheel position over time

```
>> T = 0.0001      % 100usec shaft encoder update
>> tspan = 2;      % 2 sec time vector
>> index = 1;      % 1 revolution in 2 seconds
>> [t,alpha] = alpha_gen(index,T,tspan); % generate wheel position
```

plot the wheel position over time

```
>> figure(1);
>> plot(t,alpha);
>> xlabel('time (sec)');
>> ylabel('wheel position (rad)');
>> title('wheel position in radians');
```

Given the fact that the wheel is spinning one revolution in 2 seconds, it is easy to determine the rotational rate of the wheel. Knowing the radius of the wheel it is possible to calculate the linear speed of the wheel. What are these numbers for this example?

A.2 Generate the shaft encoder output signals

```
>> Nb = 180;      % wheel pattern density (360 patches/360 deg)
>> LvL = 0;      % no noise
>> [A,B] = encoder_signals(t,alpha,Nb,LvL); % generate signals
```

plot the shaft encoder signals

```
>> figure(2);
>> plot(t,A,'.-b'); % plot A in blue
>> hold on; plot(t,B,'.-r'); hold off; % overlay B in red
>> xlabel('time (sec)'); % annotate
>> ylabel('voltage');
>> title('A(blue) B(red)');
```

A.3 Double the wheel speed by doubling **alpha** and repeat the previous part.

```
>> alpha = 2*alpha;

>> figure(1);
>> plot(t,alpha)
>> xlabel('time (sec)');
>> ylabel('wheel position (rad)');
>> title('wheel position in radians');
```

A.4 Add some noise and replot the shaft encoder output signals

```

>> LvL = 3;
>> [A,B] = encoder_signals(t,alpha,Nb,LvL); % generate signals

>> figure(2);
>> plot(t,A,'.-b'); % plot A in blue
>> hold on; plot(t,B,'.-r'); hold off; % overlay B in red
>> xlabel('time (sec)'); % annotate
>> ylabel('voltage');
>> title('A(blue) B(red)');

```

At this point, try different parameters to get a feel for how to use these matlab functions and to get a feel for how the parameters affect the generated signals.

B. Encoder Signal Generation and Processing

Write the `tic_counter.m` matlab function and then work through the following statements.

B.1 Generate encoder signals for a simple test case. A simple test case will allow results to be more easily interpreted.

```

>> T = 0.01; tspan = 1; index = 1; Nb = 4; LvL = 0;
>> [t,alpha] = alpha_gen(index,T,tspan); % generate {t,alpha}

>> figure(1); plot(t,alpha*180/pi); % plot in degrees
>> xlabel('time (sec)'); % annotate
>> ylabel('wheel position (deg)');
>> title('wheel position (degrees)');

```

The shaft encoder has 4 black patches and 4 white patches. There is no noise and the generated alpha signal is a linear counter-clockwise rotation of the wheel over 1 second, which corresponds to a rotational rate of 2π rad/sec.

B.2 Generate the shaft encoder signals and plot them

```

>> [A,B] = encoder_signals(t,alpha,Nb,LvL);

>> figure(2);
>> plot(t,A,'.-b'); % plot A in blue
>> hold on; plot(t,B,'.-r'); hold off; % overlay B in red
>> xlabel('time (sec)'); % annotate
>> ylabel('voltage');
>> title('A(blue) B(red)');

```

B.3 Find the shaft encoder counter output vector as well as the vector indicating where errors occurred.

```

>> [cV,eV,aV] = tic_counter(A,B);

```

Although `aV` represents the estimate of the wheel angle position, this is not used in the next part of the tutorial. The `alpha` estimate signal `aV` is discussed below.

Plot the tic count and show in red those that result from invalid state changes

```

>> figure(3);
>> plot(t,cV,'.-b'); % plot in blue

>> Ie = find(eV == 1);
>> hold on;
>> plot(t(Ie),cV(Ie),'r','markersize',18); % overlay errors in red
>> hold off;

```

```

>> xlabel('time (sec)');
>> ylabel('count');
>> title('tic count');

```

B.4 Estimate the wheel position from the tic count vector **cV**. This will require that you calculate the number of tic increments that occur when the wheel goes through one full revolution. Remember that this number is related to the number of black and white patches, but it is also related to the number of sensors in the shaft encoder.

```

>> ticsrev = ??? % how many tic counts are there per revolution?
>> alpha_est = cV*ticsrev; % convert from tics to angle (rad)

>> figure(1); % overlay on the plot showing {t,alpha}
>> hold on;
>> plot(t,alpha_est*180/pi,'.-r'); % plot in red and show in degrees
>> hold off;

```

You will know if **ticsrev** is correct if **figure(1)** shows two plots that have roughly the same shape and scale. The **alpha** plot will look like a line but the **alpha_est** plot will be somewhat jagged. This is normal and to be expected.

B.5 Find the alpha estimate error and plot it

```

>> errV = alpha(2:end)-alpha_est;

>> figure(4); plot(t1,errV*180/pi,'.-b');
>> xlabel('time (sec)');
>> ylabel('alpha est error (degrees)');
>> title('angle (deg)');

```

B.6 For convenience, set the figure positions as you want them so they are reproduced in that position each time you run all the preceeding lines of code

```

>> figpos('pos1');

```

then follow the instructions that show in the matlab command window.